

Multipla parametrar

Exemplet ovan använder sig av endast en typparameter, men det finns inget som hindrar att man använder två eller flera. De olika typerna är totalt oberoende av varandra. Vi kan se på en enkel klass som lagrar *par* av data, t.ex. en person och dennas socialskyddsnummer.

```
template<class T1, class T2>
class Pair {
public:
    // konstruktör
    Pair (const T1 & V1, const T2 & V2) : m_First(V1), m_Second(V2) { };

    // accessera de olika värdena
    T1 first () { return m_First; }
    T2 second () { return m_Second; };

private:
    // våra två element
    T1 m_First;
    T2 m_Second;
};
```

Denna klass är så enkel att alla metoder är definierade direkt i dess `class`-deklaration. Vi har två typer, `T1` och `T2` som definierar de datatyper som vi kan göra par av. Det är ingen skillnad vilka datatyper vi använder oss av. Vi separerar de två typerna med hjälp av ett kommatecken i `template<class T1, class T2>`-definitionen. Ett enkelt exempel som instantierar en klass som använder sig av `int-string-par`:

```
int main () {
    Pair<int,string> * Persons[2];
    int Id;
    string Name;

    // iterera och mata in data om personerna
    for ( int Index = 0; Index < 2; Index++ ) {
        cout << "Ge id: ";
        cin >> Id;
        cout << "Ge namn: ";
        cin >> Name;
        Persons[Index] = new Pair<int,string> ( Id, Name );
    }

    // skriv ut våra personer
    for ( int Index = 0; Index < 2; Index++ ) {
        cout << "Person " << Index;
        cout << ", id: " << Persons[Index]->first ()
            << ", name: " << Persons[Index]->second () << endl;

        // frigör minne
        delete Persons[Index];
    }
}
```

Programmet är ganska enkelt att förstå. Vi deklarerar här en vektor som innehåller `Pair<int,`

```
string>.
```

Det är även fullt möjligt att använda templates som har andra templates som typer. Vi kan t.ex. definiera en stack av stackar av `double` med följande anrop:

```
Stack< Stack<double> > WeirdStack;
```

Varje element i den "yttre" stacken är nu en annan stack som hanterar parametrara av typen `double`. Notera att man måste sätta minst ett mellanrum mellan de två `>` i definitionen för att särskilja definitionen från inputoperatorn `>>`.

Definiera lättare namn

Det kan ibland bli ganska svåra och oöverksådligt långa namn på datatyper. Det är enkelt att tappa bort vilket som är namnet på datatyperna och vilket är namnet på den egentliga variabeln. Vi kan ta som exempel en prioritetskö som använder sig av datatypen `Element *`. Den normala definitionen när vi vill deklarerar en variabel vore då t.ex.:

```
PriorityQueue<Element *> Queue1;
```

Vi kan med hjälp av en `typedef` göra namnet lättare att använda så vi slipper de fula `<` och `>` som följer med templateinstantieringar:

```
// definiera en lättare typ
typedef PriorityQueue<Element *> ElementQ;

// använd denna nya typ
ElementQ Queue1;
```

En hel del enklare eller hur? Detta är en av de få situationer där man normalt använder sig av `typedef`.

[Förutgående](#)
Ett konkret exempel

[Hem](#)
[Upp](#)

[Nästa](#)
Standard Template Library